STAR EAST

BEYOND THE SCRIPT:

Unlocking Quality Through Exploratory Testing

Practical Strategies and Live Demonstrations

Matt Bradley

Sr Director – Head of QA | Altus Group | UK

26	10+	100s	∞
years in testing	organizations	testers mentored	bugs found

1

About Me

Sr Director – Head of QA at Altus Group

- Matt Bradley
- Sr Director – Head of QA Altus Group
- 26 years in software testing
- Based in the UK
- Passionate about quality culture & building world class QA teams

Companies I've Worked With

Altus Group, Applied, The Pensions Regulator, Airbus Defence & Space, Mercer, rdfdigital.com, Lloyds TSB, First Title, EDS

2

What I Will Be Covering Today

By the end of this session, you'll take away new ways of working you can apply to your own companies

01
What Is Exploratory Testing?

02
Why Scripted Testing Isn't Enough

03
The Mindset of an Exploratory Tester

04
When to Use Exploratory Testing

05
Showing the Value to Stakeholders

06
Becoming a Skilled Exploratory Tester

07
Does Exploratory Testing just have to focus on software?

08
Review of some sites I looked at, and you suggest some sites for me to review

3

Who here has ever tested

without a script?

That's exploratory testing — you were already doing it!

4

What Is Exploratory Testing?

Defined, demystified, distinguished

> *"Simultaneous learning, test design, and test execution guided by a skilled tester's curiosity and judgment — not a pre-written script."*

Concurrent Design & Execute

Planning and execution happen simultaneously. You adapt instantly to what you find.

Charter-Based Sessions

Time-boxed missions guide focus: e.g. 'Explore login flows focusing on error states.'

Not Ad-Hoc : Skilled & Systematic

Testers apply smart thinking, structured approaches, and domain knowledge. This is a professional discipline.

Complements Scripted Testing

ET fills the gaps scripts miss: edge cases, emergent behavior, real-world usage patterns.

5

Exploratory Testing — Key Characteristics

Testing requirements match up all the way through a system

- Testing requirements match up all the way through a system
- Testers relying on curiosity, intuition and experience
- Find bugs scripts missed
- No predefined test cases
- Fast feedback
- Good way to find minor bugs
- Finding edge cases
- No prior knowledge of system required
- Structured but flexible testing
- Testing requirements match up to different parts of requirements
- Adaptable to complexity

6

What Exploratory Testing isn't?

- It's not the absence of documentation
- It's not unplanned or random clicking around
- It's not something anyone can do without skill
- It's not a substitute for test automation
- Not a bolt on to 'real testing'
- Not just for finding bugs

7

Why Exploratory Testing Matters

"How many times have you found a bug hours before a release just by playing with the software? Why did you miss it?"

🐛 Scripts Miss Real Bugs	👤 Users Don't Follow Test Scripts	🤔 Automation Can't Feel 'Broken'	⚡ Fast Feedback on New Features
Real defects live outside happy paths. ET finds the bugs that automation and scripted tests never reach.	Real users are curious, rushed, distracted, and make mistakes. ET mimics real human behavior.	A tester notices something 'feels wrong' even when tests pass. That human intuition is irreplaceable.	ET delivers rapid, meaningful feedback at the start of sprints — before automation is even written.

8

When and Why Should We Undertake Exploratory Testing?

Right tool, right moment

WHEN	WHY
Before your biggest client sees it	Because users don't follow test scripts
When automation passes but doubt remains	Automation can't feel broken
Every new feature, first time out	Real bugs live outside happy paths
Right after a critical bug fix	"All tests passed" does not always mean "All good"
When time is limited	ET is the highest ROI use of limited test time
Early in the dev lifecycle	Catch defects before they're expensive to fix
After a major refactor	Regression gaps are invisible to scripted suites
When a bug touches a key area	High-risk areas deserve human investigation

9

When and why should we undertake it?

- Testing requirements match up all the way through a system
- As soon as you can get an early build
- When a new feature is introduced
- As soon as you get requirements – check all areas and fields
- To provide some friendly competition into the teams
- When time is limited
- For putting the finishing touches on a release – good quality can't be spotted by automation tests
- To find high impact defects before they are harder to fix/undo
- When a major refactor has been undertaken
- It should be used continuously
- When a bug fix touches a risky or key area

10

Problems with Traditional Testing

Scripts do their job — but they're only as good as what you wrote

Scripts Only Verify Known Behavior

Manual and automated scripts confirm what was anticipated — but software fails in unexpected ways.

People Follow Scripts To The Letter

Testers become button-clickers. Valuable domain knowledge and critical thinking go unused.

Flaky Automated Tests

Intermittent failures erode trust in test results — teams start ignoring the signal.

Exploratory testing fills ALL these gaps!

Unknown Unknowns

Scripts can't test for what wasn't anticipated. The things you don't know you don't know.

Stale Test Cases

Test suites age. Outdated scripts create false confidence and miss new failure modes.

Often Time-boxed or Time-restricted

Rigid schedules prevent deep investigation. Speed is prioritised over quality insight.

11

Showing the Value — Metrics & Business Outcomes

Connect to Business Outcomes

- Stakeholders care about risk, cost, and reputation — not test coverage
- Frame exploratory testing in those terms: "We found 3 critical bugs that would have hit production" is more compelling than "we ran 40 exploratory sessions."
- Tie bugs found to their potential impact: customer churn, support tickets, regulatory risk, revenue loss.

Track & Report the Right Metrics

Bug Yield

how many defects found per session, especially compared to scripted testing.

Defect Severity

Exploratory testing tends to surface complex, high-severity bugs that scripts miss

Escaped Defects

Track how many production bugs were not caught by scripted tests but would have been caught by exploration

Coverage of Risk Areas

Use session-based test management (SBTM) charters to show which risk areas were investigated

12

Showing the Value to Stakeholders

Use Session-Based Test Management (SBTM)

SBTM gives stakeholders visibility into what would otherwise look like "ad hoc" testing. Each session has:
A **charter** (what you're explor ing and why)
A **time box**
A **debrief** (what was found, bugs, obstacles, notes)

This produces a paper trail that shows deliberate, structured thinking — not just poking around.

Show What Scripted Testing Misses

Run a direct comparison when you can:
After a scripted test pass, **run exploratory sessions** on the same features

Document what the scripts didn't catch

This is often the most persuasive **evidence**, because it makes the gap concrete

Build a Bug Portfolio Over Time

Keep a log of "**notable finds**" from exploratory testing — bugs that were **impactful, surprising**, or that scripted tests would never have caught.

Over time this becomes a **compelling evidence** base you can present in quarterly reviews or during planning conversations.

Frame it as Risk Management

Position exploratory testing as the activity that handles **unknown unknowns**.

Scripts test what you expect; exploratory testing is how you find what you **didn't think to expect**.

For stakeholders worried about **production incidents**, this framing resonates strongly.

13

Exploratory Tester's Mindset

The qualities that separate great exploratory testers from the rest

"What if" Mindset

Always ask: what happens if I do this differently? Challenge every assumption.

I Want to Break It

Think like an attacker : uncover what can go wrong before users ever see it.

I Want It to Be Perfect

Care deeply about quality. Your standards drive the depth of your exploration.

Curious

Follow threads. Anomalies are invitations to explore further, not things to ignore.

Skepticism

Don't accept 'it works on my machine.' Verify assumptions, question happy paths.

Experienced with Bad Quality

Pattern recognition built from seeing failures. You know where bugs hide.

14

Risk Intuition & Breaking Through Blocks

Over time you get to know where most bugs are found

Risk Intuition

- Some testers instinctively know where to look for bugs : this is developed intuition built over years of experience.
- Over time you get to know where most bugs are found: integrations, state transitions, error handling, data edge cases.
- Risk intuition : knowing which 20% of the code holds 80% of the defects.
- Build it by : reflecting on past bugs, reading post-mortems, studying system architecture, and lots of deliberate practice.

When You're Stuck

Change ONE variable at a time:

Same action, different data
Try boundary values, empty inputs, special characters, max lengths

Same data, different timing
Interrupt mid-action, refresh, navigate away and back

Same timing, different role
Admin vs user vs guest — same workflow, different permissions

Same role, different device
Mobile, tablet, slow network, different browsers

15

The Exploratory Testing Cycle (Cheat Sheet!)

Observe → Question → Experiment → Learn → Adjust

Observe
Notice everything — UI, timing, error messages, console output

→

Question
Ask 'What if?' at every step. Challenge assumptions & specs

→

Experiment
Design a micro-test instantly based on your question

→

Learn
What did you find? What does it tell you about the system?

→

Adjust
Guide your next test based on what you just learned

Learn → Expose Risk → Guide the Next Test

Follow signals, not scripts.

Observe → Question → Experiment → Learn → Adjust
What did I just learn — and what should I try next?

16

Exploratory Testers Cheat Sheet — What to Explore

A structured framework for knowing WHERE to focus your exploration

USERS	FLOWS & STATE	DATA & ENVIRONMENT	INTEGRATIONS
First-time use	Start → middle → end	Empty, max/min, duplicates	Third-party API failures
Rushed / distracted users	Repeat actions	Missing or shared data	Timeouts & retries
Mistakes & retries	Interruptions mid-flow	Slow network / offline	Data format mismatches
Abandoned flows	Unexpected states	Roles, devices, browsers	Concurrent calls
Accessibility needs	Session expiry	Timezone / locale edge cases	Config / env differences

17

Testing Tours — Structured Exploration Techniques

Give your exploration a mission and a method

Boundary Tour

First record, last record, middle record, max length input, empty/null, leave blank. Probe every edge of every field and list.

Interruption Tour

Cancel an action mid-flow, refresh the page halfway through, turn off laptop/PC, retry after crash, test what happens after network drop.

Bad Neighbourhood Tour

Target error messages, edge cases, ugly screens, colour contrast issues, accessibility failures, confusing labels — anywhere UX breaks down.

King's Highway Tour

Walk the primary user journey end-to-end exactly as designed — test the happy path deeply before exploring variants.

Landmark Tour

Identify key features as landmarks and navigate between them in different orders — what breaks when sequence changes?

Destroyer Tour

Actively try to cause failures: wrong data, wrong sequence, concurrent users, permission escalation attempts.

18

Powerful Questions Every Exploratory Tester Should Ask

Questions that reveal risk, quality gaps, and hidden assumptions

1. What happens if this fails in Production? Who feels the pain first?
2. Which part of this release would be hardest to fix after go-live?
3. What would keep me awake about this software/application?
4. What does 'Done' look like to different stakeholders — have we met all their expectations?
5. Who will use this in ways we didn't design for? (screen readers, colour blindness, mobile)
6. What happens if the user abandons their journey halfway through? or the application crashes?
7. What happens at the boundaries?
8. How slow is too slow? At what point does performance become a defect?
9. If time is squeezed, which bugs can we ship with?
10. What would I test first if I only had 30 minutes?
11. What surprised me and why?
12. Where does my confidence drop as I move through the system?
13. What do we always get wrong and what always fails?
14. What happens if we ignore the happy path entirely?
15. Does the user get meaningful feedback if something goes wrong?
16. How do I know this is correct without a spec?
17. What would make me lose trust in this product?
18. What would a first-time user find confusing or broken?

19

Will Automation Replace Human Testing?

A question worth addressing head-on

"Automation will replace human testing?"

— The question being asked in every boardroom right now

Automation CAN	Humans CAN
✓ Run regression tests 24/7	✓ Notice things that 'feel wrong'
✓ Execute thousands of scenarios consistently	✓ Adapt to unexpected system behaviour
✓ Detect known regressions instantly	✓ Apply domain knowledge & empathy
✓ Free testers from repetitive work	✓ Ask questions that weren't pre-programmed
✓ Report pass/fail on defined assertions	✓ Find the bugs no-one thought to write a test for

The answer: Automation and ET are partners — not competitors. Great teams need both.

20

Does Exploratory Testing Have to Focus on Software?

The principles of structured curiosity apply wherever quality matters

ET is a thinking discipline, not a software tool. The same mindset and techniques apply far beyond code.

Manufacturing & Hardware

QA engineers push products to their limits, testing under stress, real-world conditions, and beyond normal specs.

Healthcare & Clinical Processes

Exploratory testing helps medical device teams find hidden process flaws that scripted checklists never catch.

Business Process Auditing

Internal auditors go beyond checklists, digging deep to find hidden risks, gaps, and inefficiencies.

Security & Pen Testing

Ethical hackers are exploratory testers, they will keep poking the system from all angles until they find the weak spot.

Education & Training

Good course designers will not just design and sit, they will explore the content like a student and find every gap.

Documentation & Content

Technical writers use exploratory thinking to catch information gaps, confusing steps, and missing context before users suffer.

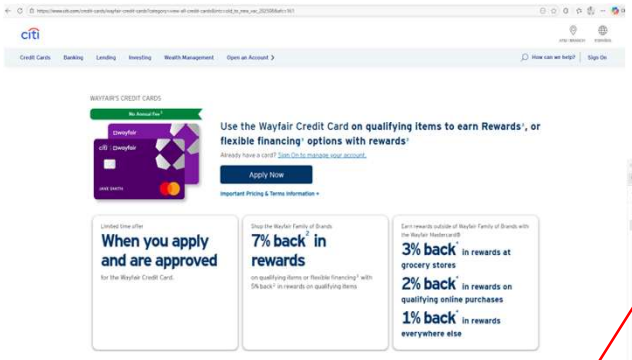
21

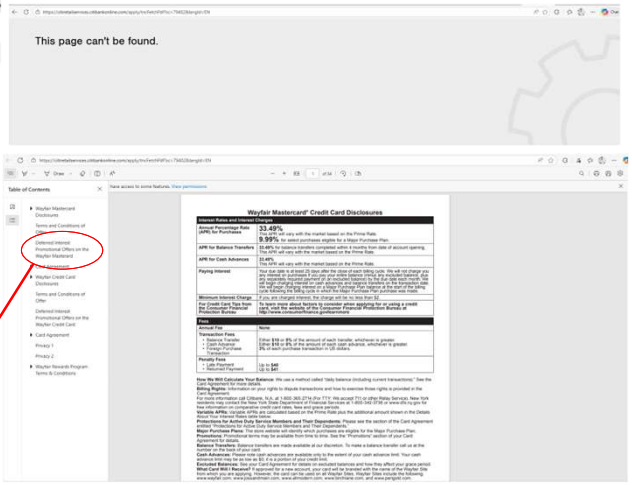
Let's review the issues I have found

22

Review of Sites I Looked At

Session format: Mission → Area Explored → Observations → Risk → Recommendation



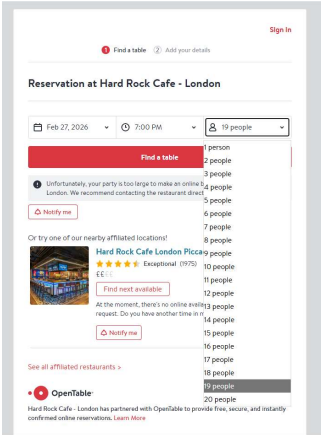


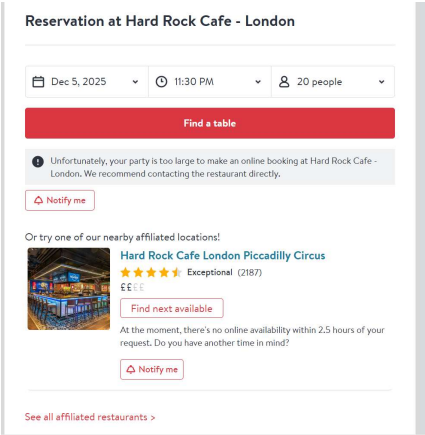
Click Important Pricing and Terms Information

23

Review of Sites I Looked At

Session format: Mission → Area Explored → Observations → Risk → Recommendation





- Why does it allow you to select up to 20 people if this is not allowed?
- Why is there no phone number to call the restaurant as it suggests calling them. Lost business!

24

Review of Sites I Looked At

Session format: Mission → Area Explored → Observations → Risk → Recommendation

Annual percentage yield (APY)

Fixed Term CD from Bank of America: View Terms and Rates

Not ready to open online?

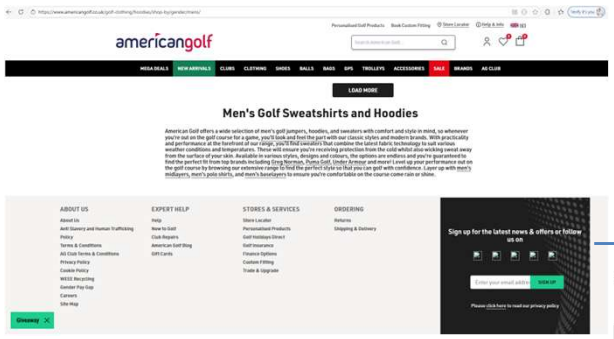
Not ready to open online?

Does not work

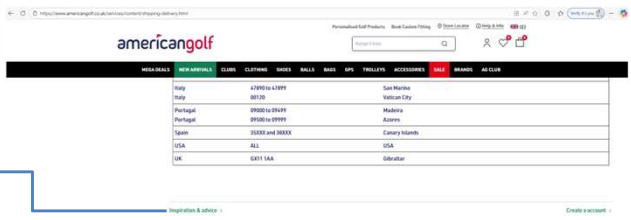
25

Review of Sites I Looked At

Session format: Mission → Area Explored → Observations → Risk → Recommendation



Broken images

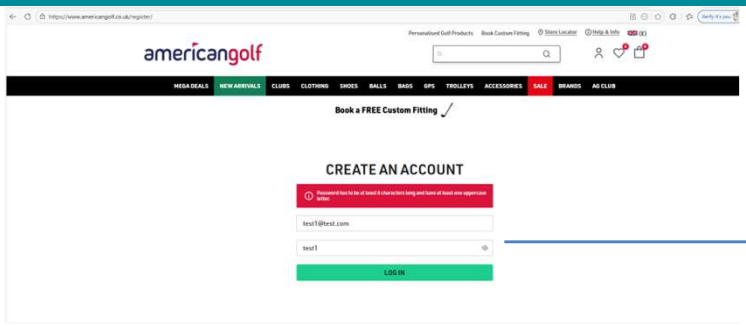


Broken Link

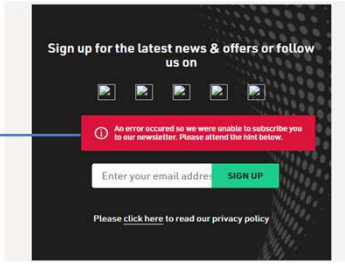
26

Review of Sites I Looked At

Session format: Mission → Area Explored → Observations → Risk → Recommendation



I am creating an account, not logging in

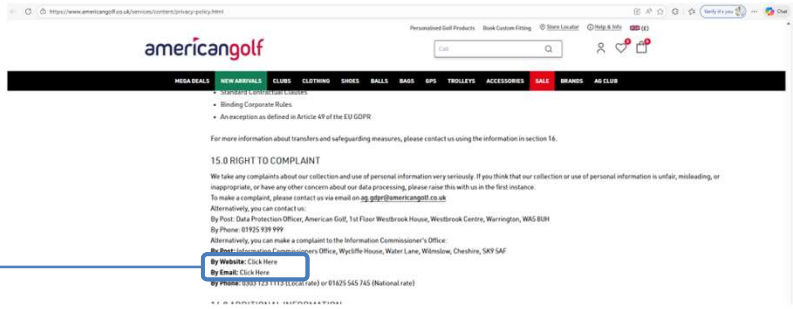


What does "Please attend the hint below" mean?

27

Review of Sites I Looked At

Session format: Mission → Area Explored → Observations → Risk → Recommendation

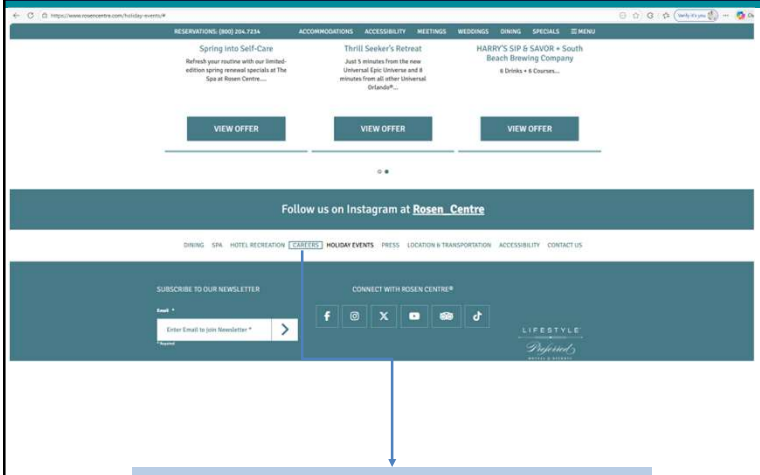


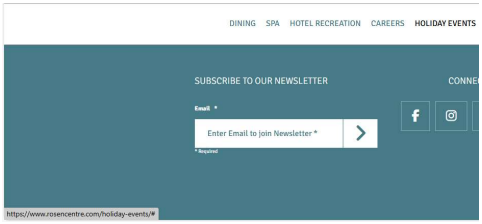
No links to click

28

Review of Sites I Looked At

Session format: Mission → Area Explored → Observations → Risk → Recommendation





Careers link will refresh to the page you are on

29

You Suggest Sites for Me to Review

Live collaborative exploratory testing — audience-driven

Suggest Any Public Site

Suggest any publicly accessible site. We'll define a charter together and run a live 10-minute ET session — observing, noting, and debriefing in real time.

30

Key Takeaways

ET is a professional discipline

Not ad hoc, not random — structured curiosity backed by heuristics, models, and 26 years of experience.

Scripts have hard limits

The best teams use both scripted AND exploratory testing, each where it excels. They are partners.

Mindset is everything

Curiosity, scepticism, and risk intuition compound over time. Invest in developing your ET skills.

ET applies beyond software

Manufacturing, healthcare, security, documentation — anywhere quality matters, ET thinking adds value.

Make your work visible

Session reports, risk maps, and business-framed findings turn ET from 'playing around' into a valued practice.

31

Exploratory Testing isn't about finding more bugs – it's about finding the right problems before users do

Thank you — keep exploring!

bradleymattj@yahoo.com
+44 (0)7765005332

32